Intelligent Multimodal Search Engine

Technical Report & Architecture Overview

January 26, 2026

Abstract

This report details the development of an AI-powered multimodal search engine designed to solve the semantic and visual limitations of traditional e-commerce search. By integrating Qdrant for vector storage and Groq (Llama-3) for intent analysis, the system achieves high-precision retrieval through a hybrid architecture of vector similarity and strict metadata filtering.

Contents

---

1 Idea

We developed a **Multimodal Search Engine** designed to create a smarter e-commerce experience. Using **RAG (Retrieval-Augmented Generation)**, the system goes beyond simple keyword matching; it searches for products based on natural language, visual cues, and current market trends or reviews. To ensure relevance, we implemented intelligent filtering that respects the user's budget and prevents common errors (such as confusing visually similar but unrelated items like vitamins and jeans). Finally, every search result includes an **LLM-generated explanation**, providing users with transparency on exactly why a specific product was recommended.

2 Architecture & Qdrant Integration

The system follows a **Hybrid Search Architecture** that combines vector similarity with deterministic logic.

2.1 A. Data Layer (Qdrant Integration)

- **Collection Name:** `Qdrant_db`
- **Vector Configuration:** multi-dimensional vectors using Cosine Similarity.
- **Payload Storage:** We store the vector embeddings alongside a rich metadata payload: `price`, `category`, `image_url`, `brand`, and `rating`.
- **Why Qdrant?** Qdrant was selected for its robust handling of **Hybrid Filtering**. We do not merely search for vectors; we search for *vectors that strictly match specific metadata conditions* (specifically Category and Price constraints).

2.2 B. Processing Layer (The "Brain")

1. **Query Intent Analysis (Groq/Llama-3):** Before searching, the user's query is analyzed by an LLM to extract structured data.
   Example: Query: "Jeans under 200" → Intent: {`Category: "Jeans", Budget: 200`}.
2. **Embedding (CLIP):** The search text is converted into a vector using the `clip-ViT-B-32` model, allowing it to match both text and image vectors in the database.
3. **Retrieval:** Qdrant retrieves the nearest neighbors that *also* satisfy the Intent Filters identified in Step 1.
4. **Explanation:** The retrieved items are passed back to the LLM to generate a natural language "sales pitch" explaining why the item fits the user's specific request.

3 Data Pipeline

The data ingestion process ensures the database is multimodal-ready. The pipeline performs the following steps:

1. **Raw Input:** A CSV dataset containing approximately 2,000 products, including fields for Name, Description, Price, and Image URLs.
2. **Context Enrichment:** The Pandas library is used to merge product descriptions with aggregated customer reviews, creating a rich `ai_context` field for better semantic understanding.
3. **Visual Encoding Strategy:**
   - The pipeline attempts to download the product image from the URL.
   - **Success:** The image is passed through CLIP to generate a visual vector.
   - **Fallback:** If the image link is broken or timed out, the textual description is encoded into a vector instead.
4. **Upsert:** The resulting vectors and metadata payloads are batch-uploaded to the local Qdrant instance.

4 Project Timeline

4.1 Phase 1: Foundation (Completed)

Data Ingestion: Successfully cleaned and uploaded 2,000+ SKUs to Qdrant with valid vector embeddings.

Vector Search: Implemented CLIP-based semantic search allowing text-to-image matching.

Hybrid Filtering: Solved accuracy issues (e.g., distinguishing Vitamins from Jeans) by implementing LLM-based category detection.

AI Explanations: Integrated Groq API to provide context-aware explanations for every search result.

- **Image-to-Image Search:** Implement a feature allowing users to upload a photo to find visually similar products (the backend currently supports this).

4.2 Phase 2: Future Work (To Do)

- **Frontend Development:** Build a user interface using Streamlit or React to allow visual interaction.
- **AI Assistant Chatbot:** Integrate a conversational agent designed to solve technical problems and provide operational guidance. This assistant will help users troubleshoot issues in real-time and navigate the system's features effectively.